📌 Part 1: Performance Diagnosis

This section presents the performance analysis of selected SQL queries executed on a dataset containing over 100,000 records. The objective is to identify potential inefficiencies using execution time measurements and query execution plans obtained through the EXPLAIN command.

Query 1: Total Enrollments per Month

SQL Query:

```
SELECT
    YEAR(enrollmentDate) AS year,
    MONTH(enrollmentDate) AS month,
    COUNT(*) AS total_enrollments
FROM enrollments
GROUP BY YEAR(enrollmentDate), MONTH(enrollmentDate)
ORDER BY year, month;
```

Execution Time:
0.313 seconds

1 16:13:53 SELECT YEAR(enrollmentDate) AS year, MONTH(enrollmentDate) AS month, COUNT(*) AS total_enrollme... 676 row(s) returned 0.313 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	enrollments	NULL	index	idx_enrollment_date	idx_enrollment_date	4	NULL	40352	100.00	Using index; Using temporary

The query executes relatively quickly; however, the use of YEAR() and MONTH() functions on the enrollmentDate column prevents the database from utilizing any index on that column. As a result, the query performs a full table scan (type: ALL), which may lead to performance degradation as the dataset grows.

Query 2: Number of Students per Course

SQL Query:

```
SELECT
    c.courseName,
    COUNT(e.studentID) AS total_students
FROM enrollments e
JOIN courses c ON e.courseID = c.courseID
GROUP BY c.courseName
ORDER BY total_students DESC;
```

Execution Time:
1.203 seconds

2 16:13:53 SELECT c.courseName, COUNT(e.studentID) AS total_students FROM enrollments e JOIN courses c ON e.c... 1000 row(s) returned 1.203 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	c	NULL	ALL	PRIMARY	NULL	NULL	NULL	10130	100.00	Using temporary; Using filesort
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	NULL

This query has the highest execution time among all tested queries. The EXPLAIN plan indicates a full table scan and lack of index usage (key: NULL). Additionally, grouping by courseName, which is a non-indexed text field, increases processing time and reduces efficiency.

Query 3: Instructor Workload

SQL Query:

```
SELECT
   i.name AS instructor_name,
   COUNT(e.studentID) AS total_students
FROM enrollments e
JOIN courses c ON e.courseID = c.courseID
JOIN instructors i ON c.instructorID = i.instructorID
GROUP BY i.name
ORDER BY total_students DESC;
```

Execution Time:
0.734 seconds

3 16:13:54 SELECT i.name AS instructor_name, COUNT(e.studentID) AS total_students FROM enrollments e JOIN cours... 1000 row(s) returned 0.734 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	i	NULL	ALL	PRIMARY	NULL	NULL	NULL	2000	100.00	Using temporary; Using filesort
1	SIMPLE	c	NULL	ref	PRIMARY,instructorID	instructorID	5	student_management_systemDB.i.instructorID	5	100.00	Using index
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	NULL

This query involves multiple joins, which increases computational cost. Although the join columns (courseID and instructorID) are indexed, the query still performs relatively slower due to the use of multiple joins and grouping by instructor name, which is a text-based field. Grouping by a non-primary key column increases aggregation overhead and reduces efficiency compared to grouping by an indexed identifier.

Query 4: Average Students per Course

SQL Query:

```
SELECT
   AVG(student_count) AS avg_students_per_course
FROM (
   SELECT
      courseID,
      COUNT(studentID) AS student_count
   FROM enrollments
   GROUP BY courseID
) AS course_counts;
```

Execution Time:
0.375 seconds

4 16:13:55 SELECT AVG(student_count) AS avg_students_per_course FROM (SELECT courseID, COUNT(s... 1 row(s) returned 0.375 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	40352	100.00	NULL
2	DERIVED	enrollments	NULL	index	courseID	courseID	5	NULL	40352	100.00	NULL

This query performs efficiently due to its use of a subquery that aggregates data by courseID, which is likely indexed. It does not involve joins or complex operations, making it relatively optimized even before improvements.

Query 5: Courses with High Enrollment

SQL Query:

```
SELECT
   c.courseName,
   COUNT(e.studentID) AS total_students
FROM enrollments e
JOIN courses c ON e.courseID = c.courseID
GROUP BY c.courseName
HAVING COUNT(e.studentID) > 5
ORDER BY total_students DESC;
```

Execution Time:
0.453 seconds

5 16:13:56 SELECT c.courseName, COUNT(e.studentID) AS total_students FROM enrollments e JOIN courses c ON e.c... 1000 row(s) returned 0.453 sec / 0.204 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	c	NULL	ALL	PRIMARY	NULL	NULL	NULL	10130	100.00	Using temporary; Using filesort
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	NULL

This query uses a HAVING clause combined with grouping on a non-indexed column (courseName). Similar to Query 2, the lack of indexing results in full table scans and inefficient grouping operations, which may impact performance with larger datasets.

📌 Part 2: Optimization Implementation

This section presents the optimization techniques applied to improve the performance of the SQL queries identified in Part 1. The goal was to reduce execution time and enhance query efficiency through indexing and query refactoring.

Indexing Implementation

Upon analyzing the database schema, it was observed that several columns commonly used in JOIN operations such as `studentID`, `courseID`, and `instructorID` were not initially optimized for faster access. To address this, composite indexes were created to improve the performance of JOIN queries.

Additional indexes were implemented on frequently used columns in filtering, searching, and sorting operations, including attributes like names, email, grade, and date-related fields. Composite indexing was also applied to combinations of columns often used together in conditions and ordering.

```
--  ENROLLMENTS
--
CREATE INDEX idx_enrollments_student_course ON enrollments(studentID,
courseID);
CREATE INDEX idx_enrollments_grade ON enrollments(grade);
CREATE INDEX idx_enrollments_date_grade ON enrollments(enrollmentDate,
grade);

-- STUDENTS
CREATE INDEX idx_students_lastname ON students(lastName);
CREATE INDEX idx_students_firstname ON students(firstName);
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_students_name ON students(lastName, firstName);

--
-- COURSES
--
CREATE INDEX idx_course_instructor ON courses(courseID, instructorID);
CREATE INDEX idx_courses_name ON courses(courseName);

--
-- AUDIT LOGS
--
CREATE INDEX idx_audit_action ON audit_logs(action);
CREATE INDEX idx_audit_tableName ON audit_logs(tableName);
CREATE INDEX idx_audit_actionTime ON audit_logs(actionTime);
CREATE INDEX idx_audit_table_action ON audit_logs(tableName, action);
```

```
-- DIM TABLES
--
CREATE INDEX idx_dim_student_id ON dim_student(studentID);
CREATE INDEX idx_dim_student_email ON dim_student(email);
CREATE INDEX idx_dim_student_name ON dim_student(lastName, firstName);

CREATE INDEX idx_dim_course_id ON dim_course(courseID);
CREATE INDEX idx_dim_course_name ON dim_course(courseName);
CREATE INDEX idx_dim_course_instructor ON dim_course(instructorID);

CREATE INDEX idx_dim_instructor_id ON dim_instructor(instructorID);
CREATE INDEX idx_dim_instructor_name ON dim_instructor(name);
CREATE INDEX idx_dim_instructor_email ON dim_instructor(email);

CREATE INDEX idx_dim_date_full ON dim_date(fullDate);
```

Query 1: Total Enrollments per Month

SQL Query:

```
SELECT
   YEAR(enrollmentDate) AS year,
   MONTH(enrollmentDate) AS month,
   COUNT(*) AS total_enrollments
FROM enrollments
GROUP BY year, month
ORDER BY year, month;
```

Execution Time:
0.235 seconds

1 19:06:59 SELECT YEAR(enrollmentDate) AS year, MONTH(enrollmentDate) AS month, COUNT(*) AS total_enrollments FROM enrollments GROUP BY year, month ORDER BY year, month LIMIT 0, 1000 676 row(s) returned 0.235 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	enrollments	NULL	index	idx_enrollment_date	idx_enrollment_date	4	NULL	40352	100.00	Using index; Using temporary; Using filesort

This query shows slight improvement after optimization. An index was added to the `enrollmentDate` column, reducing scan cost. However, the use of YEAR() and MONTH() functions still limits full index utilization. Despite this, the query performs better than the original due to reduced redundancy in the GROUP BY clause.

Query 2: Number of Students per Course

SQL Query:

```
SELECT
    c.courseName,
    COUNT(*) AS total_students
FROM enrollments e
JOIN courses c
    ON e.courseID = c.courseID
GROUP BY c.courseID, c.courseName
ORDER BY total_students DESC;
```

Execution Time:
0.265 seconds

2 19:07:00 SELECT c.courseName, COUNT(*) AS total_students FROM enrollments e JOIN courses c ON e.courseID = c.courseID GROUP BY c.courseID, c.courseName ORDER BY total_students DESC LIMIT 0, 1000 1000 row(s) returned 0.265 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	c	NULL	ALL	PRIMARY	NULL	NULL	NULL	10130	100.00	Using temporary; Using filesort
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	Using index

This query performs slowly due to grouping on a text-based column (courseName), which increases aggregation cost. The join operation also processes a large dataset from the enrollments table. Since courseName is not indexed, MySQL performs additional work to group and sort results, leading to higher execution time.

Query 3: Instructor Workload

SQL Query:

```
SELECT
    i.name AS instructor_name,
    COUNT(*) AS total_students
FROM instructors i
JOIN courses c
    ON i.instructorID = c.instructorID
JOIN enrollments e
    ON c.courseID = e.courseID
GROUP BY i.instructorID, i.name
ORDER BY total_students DESC;
```

Execution Time:
0.250 seconds

3 19:07:00 SELECT i.name AS instructor_name, COUNT(*) AS total_students FROM instructors i JOIN courses c ON i.instructorID = c.instructorID JOIN enrollments e ON c.courseID = e.courseID GROUP BY i.instructorI... 1000 row(s) returned 0.250 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	i	NULL	ALL	PRIMARY	NULL	NULL	NULL	2000	100.00	Using temporary; Using filesort
1	SIMPLE	c	NULL	ref	PRIMARY,instructorID	instructorID	5	student_management_systemDB.i.instructorID	5	100.00	Using index
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	Using index

This query performs more efficiently after optimization. Grouping by `instructorID` instead of only the instructor name reduces the cost of string comparison. Existing indexes on join columns are effectively utilized, resulting in faster join operations and improved overall performance.

Query 4: Average Students per Course

SQL Query:

```
SELECT
   AVG(student_count) AS avg_students_per_course
FROM (
   SELECT
      courseID,
      COUNT(*) AS student_count
   FROM enrollments
   GROUP BY courseID
) AS course_counts;
```

Execution Time:
0.234 seconds

4 19:07:00 SELECT AVG(student_count) AS avg_students_per_course FROM (SELECT courseID, COUNT(*) AS student_count FROM enrollments GROUP BY courseID) AS course_counts LIMIT 0, 1000 1 row(s) returned 0.234 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	40352	100.00	NULL
2	DERIVED	enrollments	NULL	index	courseID	courseID	5	NULL	40352	100.00	Using index

Since the query does not involve joins or complex filtering, it remains relatively efficient compared to the other queries.

Query 5: Courses with High Enrollment

SQL Query:

```
SELECT
    c.courseName,
    COUNT(*) AS total_students
FROM enrollments e
JOIN courses c
    ON e.courseID = c.courseID
GROUP BY c.courseID, c.courseName
HAVING total_students > 5
ORDER BY total_students DESC;
```

Execution Time:
0.281 seconds

5 19:07:00 SELECT c.courseName, COUNT(*) AS total_students FROM enrollments e JOIN courses c ON e.courseID = c.courseID GROUP BY c.courseID, c.courseName HAVING total_students > 5 ORDER BY total_stud... 1000 row(s) returned 0.281 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	c	NULL	ALL	PRIMARY	NULL	NULL	NULL	10130	100.00	Using temporary; Using filesort
1	SIMPLE	e	NULL	ref	courseID	courseID	5	student_management_systemDB.c.courseID	4	100.00	Using index

This query shows improved performance after optimization. Grouping by `courseID` instead of `courseName` reduces overhead and allows better use of indexing. The HAVING clause is applied more efficiently after aggregation, resulting in faster filtering of high-enrollment courses.